



JSON-RPC-LD: SEMANTIC RPC ARCHITECTURE

Bridging the gap between lightweight RPC
and the Semantic Web through strict
validation and explicit context.



ANALYSIS & INSIGHTS

December 2025

JSON-RPC-LD TRANSFORMS SIMPLE RPC INTO SEMANTICALLY-RICH COMMUNICATION

02

■ Core Innovation

All params/results include @context, turning plain RPC into Linked Data with global semantics.

■ Design Philosophy

Treats SHACL shapes as formal contracts between client and server, with validation enforced at every interaction point.

■ Key Distinction

Intentionally does NOT support unconstrained JSON-RPC; this choice favors semantic integrity over broad compatibility.

**"Simplicity
with semantic
expressiveness
and robust
validation."**

The JSON-RPC-LD Promise

FOUR-LAYER ARCHITECTURE ENABLES SEMANTIC VALIDATION

03

04 Validation Layer (SHACL)

Defines constraints on structure, types, cardinality, and values, enabling runtime validation and contract verification.

03 Semantic Layer (JSON-LD)

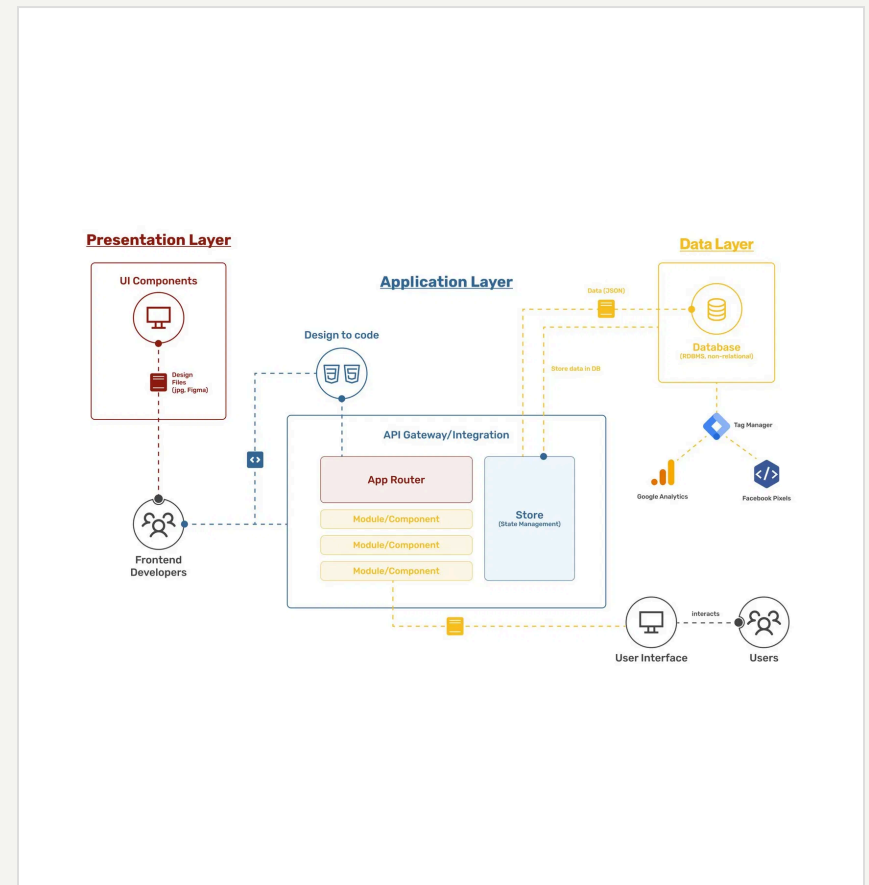
Maps JSON keys to globally unique IRIs via @context, transforming data into self-describing Linked Data.

02 RPC Layer (JSON-RPC 2.0)

Manages the request-response cycle, method invocation, parameter passing, and standard error handling.

01 Transport Layer

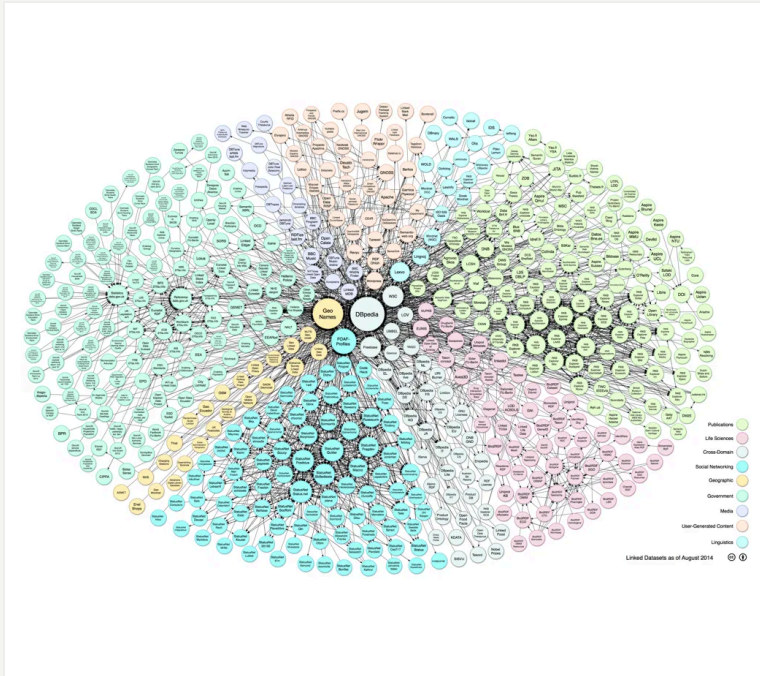
Underlying mechanism (HTTP, WebSockets) handling raw message delivery between client and server.



Conceptual Model of Layered Architecture

MANDATORY CONTEXT ENSURES SEMANTIC EXPLICITNESS

04



Visualizing the global semantic web where every data point is uniquely identified.

■ Eliminates Ambiguity

Every data element has explicit semantic meaning through IRI mapping, preventing misinterpretation between implementations.

■ Contract-First Design

Encourages defining semantic contracts before implementation, ensuring all parties agree on the data structure and meaning.

■ No Implicit Semantics

Unlike REST APIs where field meanings are often implicit, JSON-RPC-LD makes all semantics explicit and machine-readable.

EXAMPLE: EXPLICIT SEMANTIC MAPPING

```
{
  "@context": {
    "User": "http://example.org/User",
    "id": "http://example.org/User#id"
  }
}
```

BIDIRECTIONAL VALIDATION ENFORCES CONTRACTS AT EVERY BOUNDARY

05

INBOUND VALIDATION

Protects servers from malformed requests and ensures business logic receives valid data conforming to expected constraints.

OUTBOUND VALIDATION

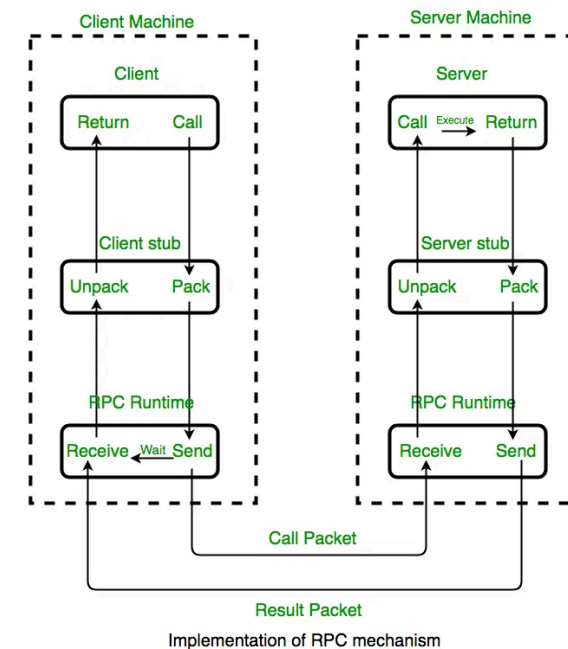
Catches implementation errors before responses reach clients, ensuring servers produce valid responses.

COMPILE-TIME SAFETY

In statically-typed languages like TypeScript, many SHACL constraints are enforced at compile time.

RUNTIME ENFORCEMENT

Complex constraints like value ranges and patterns are validated at runtime with detailed error reporting.



Validation occurs at the Client Stub and Server Stub boundaries

LANGUAGE-SPECIFIC IMPLEMENTATIONS LEVERAGE NATIVE TYPE SYSTEMS

06

RUBY

w/ Sorbet

Generates Ruby classes with Sorbet type annotations, providing static type checking in a traditionally dynamic language environment.

PYTHON

w/ mypy & Pydantic

Creates type-hinted classes with robust runtime validation, fully integrating with Python's modern type checking ecosystem.

TYPESCRIPT

w/ Zod or io-ts

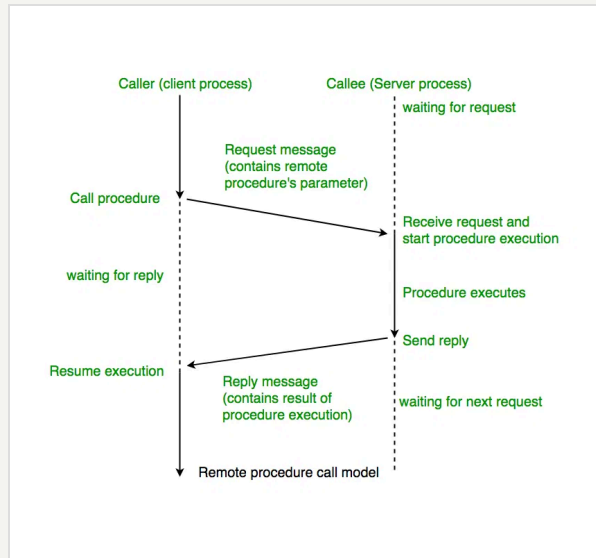
Produces TypeScript interfaces for compile-time safety combined with runtime validation libraries to ensure end-to-end correctness.

SHARED PRINCIPLE

Each implementation leverages the idiomatic type-checking tools of its language while generating consistent validation logic derived directly from SHACL shapes.

JSON-RPC-LD VS GRPC: SEMANTIC INTEROPERABILITY VS PERFORMANCE

07



Both protocols share the fundamental Remote Procedure Call model, but diverge significantly in their architectural priorities and data handling mechanisms.

gRPC

PERFORMANCE

Binary Efficiency

Uses Protocol Buffers for compact, high-speed binary serialization ideal for low-latency communication.

TYPING

Language-Specific

Strong typing via code generation from .proto files, mapping directly to native language structures.

BEST FIT

Internal Microservices

Optimized for high-throughput internal traffic where semantic context is implicit or shared.

JSON-RPC-LD

INTEROPERABILITY

Semantic Richness

Uses JSON-LD and global IRIs to ensure data is self-describing across organizational boundaries.

TYPING

Semantic Contracts

Validates against SHACL shapes, enforcing constraints based on meaning rather than just structure.

BEST FIT

Data Integration

Ideal for public APIs, knowledge graphs, and systems requiring explicit semantic understanding.

JSON-RPC-LD VS OPENAPI: VALIDATION & SEMANTICS

08

VALIDATION DEPTH

OPENAPI (JSON SCHEMA)

Focuses on structure, data types, and basic formatting. Limited ability to express complex logical dependencies.

JSON-RPC-LD (SHACL)

Supports advanced constraints: logical operators (AND/OR/NOT), cross-property comparisons, and custom SPARQL rules.

SEMANTIC RICHNESS

OPENAPI

Semantics are often implicit or limited to human-readable descriptions. Lacks native global interoperability.

JSON-RPC-LD

Uses JSON-LD contexts to map data to global IRIs, enabling data understanding across organizational boundaries.

ECOSYSTEM

OPENAPI

Mature, industry-standard ecosystem with extensive tooling for documentation, testing, and code generation.

JSON-RPC-LD

Emerging ecosystem. Tooling is growing but requires semantic web expertise to leverage fully.

09



JSON-LD contexts enable data understanding across organizations using shared vocabularies like Schema.org or domain-specific ontologies.

SHACL's expressive constraint language catches errors early and ensures data quality at API boundaries.

Combination of JSON-LD contexts and SHACL shapes creates machine-readable contracts that serve as executable documentation.

Semantic contracts are language-agnostic, enabling clients and servers in different languages to communicate with guaranteed compatibility.

Provides compile-time and runtime type safety while maintaining the flexibility of JSON-based communication.

KEY CHALLENGES: COMPLEXITY & ECOSYSTEM MATURITY

10

STRATEGIC HURDLES

STEEP LEARNING CURVE

Developers must master JSON-LD, SHACL, and RDF concepts in addition to standard RPC patterns, significantly increasing onboarding time.

SEMANTIC MODELING EXPERTISE

Effective implementation requires thoughtful vocabulary selection and ontology design, demanding specialized semantic web skills.

TECHNICAL HURDLES

PERFORMANCE OVERHEAD

Runtime validation and JSON-LD processing introduce computational costs that exceed simple JSON parsing or binary protocols.

TOOLING MATURITY

The ecosystem is nascent compared to gRPC or OpenAPI; libraries and code generation tools are still emerging and less battle-tested.

EVOLUTION MANAGEMENT

Strict semantic contracts make API evolution more rigid; maintaining backward compatibility requires careful versioning strategies.

RECOMMENDED USE CASES FOR JSON-RPC-LD

11

SEMANTIC WEB APPLICATIONS

Systems integrating directly with RDF databases, SPARQL endpoints, or requiring reasoning capabilities over API data.

CROSS-ORGANIZATION EXCHANGE

APIs where multiple organizations need a shared, explicit understanding of data semantics beyond simple field names.

HIGHLY REGULATED DOMAINS

Industries like healthcare or finance requiring rigorous validation and auditable, machine-readable data contracts.

KNOWLEDGE GRAPHS & AI

Applications feeding data into knowledge graphs or AI systems that benefit from explicit semantic relationships and types.

NOT RECOMMENDED FOR

High-performance microservices, simple CRUD APIs, or teams without semantic web expertise (use gRPC or OpenAPI instead).

CONCLUSION: SEMANTIC RPC FOR INTEROPERABLE SYSTEMS

12

ARCHITECTURAL INNOVATION

The four-layer architecture successfully combines transport, RPC, semantic, and validation layers into a cohesive, robust protocol.

STRATEGIC TRADE-OFFS

Accepts increased complexity and performance overhead in exchange for guaranteed semantic interoperability and contract enforcement.

FUTURE POTENTIAL

Success depends on the maturity of code generation tools, community adoption, and the growing need for intelligent, interconnected systems.

KEY TAKEAWAY

JSON-RPC-LD is not a universal replacement for gRPC or OpenAPI, but a **specialized tool for systems where semantic clarity and global interoperability are paramount.**